

Functional Programming Exam:

Thursday, May 28, 2026

- The exam duration is five hours.
- There are four questions. To obtain full marks, you must answer all subquestions satisfactorily.
- You are allowed to use books, lecture notes, lecture slides, hand-ins, solutions to assignments, calculators, computers, software, etc. during the examination. This includes F# compilers and IDEs.
- You may **NOT** use the internet in any way other than WiseFlow for the duration of the exam and you must be connected to WiseFlow, and its monitoring software, at all times.
- You **MUST** use the ITU++ WiFi network. Make sure to forget any networks that you may accidentally join by mistake and turn off any mobile devices so that you don't accidentally join a hot spot.
- You may **NOT** use any form of code generators like Visual Studio CoPilot, ChatGPT or equivalent. All code you hand in must either be in the template we provide, or written by you yourself. The use of any such tool is grounds for disciplinary action. Standard auto-completion like IntelliSense is ok.
- You may **NOT** communicate with anyone else during the exam, which includes using chat clients, phones, hand gestures, smoke signals, etc. The exception is the invigilators if you have a question.
- All forms of software that can be used to communicate, like Steam, Messenger, Discord, Teams, etc, and any code generation tools (CoPilot, LLMs like ChatGPT, etc) must be completely switched off for the entire exam. It's not enough not to use them, they must be switched off completely. To be safe, restart your computer before the exam and make sure that no banned programs are running.
- You are (unless otherwise instructed) allowed to use the .NET library including the modules described in the book, e.g., `List`, `Set`, `Map`, etc.
- If a question requires you to define a function, then you may (unless otherwise instructed)
 - use that function in subsequent subquestions, even if you have not managed to define it. Providing the signature of the missing function will help in such cases.
 - define as many helper functions as you want, but you must define the required function so that it has exactly the intended type and effect, and the helper functions must satisfy the constraints required by the question. A tail-recursive assignment may, for instance, be coded as a non-recursive main function that calls a tail-recursive auxiliary function.
- Unless explicitly stated, you must provide functional solutions without side effects.
- You are required to use the provided code project `FPEXAM2026` as a basis for your submission and you should **only hand in** the `Exam.fs` file. No other file, and **absolutely not a zip file of the entire project**. The project includes everything you need to run as an independent project. Any helper functions that we provide in the `Exam.fs` file may also be part of your submission.
- If an assignment asks that you write a function `isEven : int -> bool`, for instance, then there is a corresponding `let isEven _ = failwith "not implemented"` in `Exam.fs`. You may change these functions (changing a `let` to a `let rec` for instance) as long as their signatures match those given in the assignment. In this case that could be `let isEven x = x % 2 = 0`.

You MUST include explanations and comments to support your solutions for the questions that require them. You simply write them as comments around your code.

Your entire exam hand-in MUST be made by yourself and yourself alone It is illegal to make the exam answers as group work or to enlist the help of others in any way. This includes using solutions or code found online, or tools that write code for you (such as CoPilot or ChatGPT).

Your solution MUST compile. We reserve the right to fail any submission that does not meet this minimal requirement.

1: Parametric Lucas numbers (25%)

Parametric Lucas numbers create a recursive sequence that can be used to quickly approximate square roots of specific numbers. We put all the formulas you need for all sub-questions in the front to give you one point of reference.

The n th Lucas number for a parameter k , written as L_k^n is defined as

$$L_k^n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ k \cdot L_k^{n-1} + L_k^{n-2} & \text{otherwise} \end{cases}$$

This formula should look familiar. When $k = 1$, this recurrence produces the Fibonacci sequence. The sequence obtained when $k = 2$ is called the Pell sequence. Here are the sequences obtained when k is equal to 1, 2, and 5.

- $L_1^0, \dots, L_1^9 = 0, 1, 1, 2, 3, 5, 8, 13, 21, 34$
- $L_2^0, \dots, L_2^9 = 0, 1, 2, 5, 12, 29, 70, 169, 408, 985$
- $L_5^0, \dots, L_5^9 = 0, 1, 5, 26, 135, 701, 3640, 18901, 98145, 509626$

We call this constant parameter k the base of the sequence. Depending on this base k , the n th approximation of the square root of $k^2 + 4$ can be approximated using the formula

$$\sqrt{k^2 + 4} \approx 2 \cdot \frac{L_k^n}{L_k^{n-1}} - k$$

For instance, using the sixth Lucas number we get the following results when k is equal to 1, 2, and 5.

$$\begin{aligned} \sqrt{1^2 + 4} &= \sqrt{5} = 2.23606 \approx 2.2 = 2 \cdot \frac{8}{5} - 1 = 2 \cdot \frac{L_1^6}{L_1^5} - 1 \\ \sqrt{2^2 + 4} &= \sqrt{8} = 2.82843 \approx 2.82759 = 2 \cdot \frac{70}{29} - 2 = 2 \cdot \frac{L_2^6}{L_2^5} - 2 \\ \sqrt{5^2 + 4} &= \sqrt{29} = 5.38516481 \approx 5.385164051 = 2 \cdot \frac{3640}{701} - 5 = 2 \cdot \frac{L_5^6}{L_5^5} - 5 \end{aligned}$$

Question 1.1

Create a recursive, but not tail-recursive function `lucas_number` of type `int -> int -> int` which, given a base k and an integer n , computes the Lucas number L_k^n .

You do not have to worry about performance.

Examples:

```
> [0..9] |> List.map (lucas_number 1)
- val it: int list = [0; 1; 1; 2; 3; 5; 8; 13; 21; 34]

> [0..9] |> List.map (lucas_number 2)
- val it: int list = [0; 1; 2; 5; 12; 29; 70; 169; 408; 985]

> [0..9] |> List.map (lucas_number 5)
- val it: int list = [0; 1; 5; 26; 135; 701; 3640; 18901; 98145; 509626]
```

Question 1.2

Create a tail-recursive function `lucas_number_acc`, using an accumulator (not continuations), that produces the same result as `lucas_number` from Q1.1.

For full credit, the function must run in linear time.

Question 1.3

Create a function `sqrt_approx` of type `int -> int -> float` which, given a base `k` and an integer `n`, returns the n th approximation of the square root $\sqrt{k^2 + 4}$ as described above.

Do not get stuck if your floating-point results do not line up exactly. Floating-point computations may vary slightly depending on evaluation order, but the results should be close.

Examples:

```
> [2..6] |> List.map (sqrt_approx 1)
- val it: float list = [1.0; 3.0; 2.0; 2.333333333; 2.2]

> [2..6] |> List.map (sqrt_approx 2)
- val it: float list = [2.0; 3.0; 2.8; 2.833333333; 2.827586207]

> [2..6] |> List.map (sqrt_approx 5)
- val it: float list = [5.0; 5.4; 5.384615385; 5.385185185; 5.385164051]
```

Question 1.4

Create a function `approx_steps_needed` of type `int -> float -> int` which, given a base `k` and a floating point number `epsilon`, returns the smallest $n > 1$ such that $|\sqrt{k^2 + 4} - \text{sqrt_approx } k \ n| < \text{epsilon}$

Examples:

```
> approx_steps_needed 2 0.001
- val it: int = 6

> approx_steps_needed 2 0.0001
- val it: int = 8
```

Question 1.5

Create function `lucas_seq` of type `int -> seq<int>` which, given a base `k`, lazily generates the infinite parametric Lucas sequence $L_k^0, L_k^1, L_k^2, \dots$ for base `k`.

For full credit, your implementation should avoid recomputing previously generated Lucas numbers.

Examples:

```
> 1 |> lucas_seq |> Seq.take 10 |> Seq.toList
- val it: int list = [0; 1; 1; 2; 3; 5; 8; 13; 21; 34]

> 2 |> lucas_seq |> Seq.take 10 |> Seq.toList
- val it: int list = [0; 1; 2; 5; 12; 29; 70; 169; 408; 985]

> 5 |> lucas_seq |> Seq.take 10 |> Seq.toList
- val it: int list = [0; 1; 5; 26; 135; 701; 3640; 18901; 98145; 509626]
```

2: Code comprehension (25%)

Consider the following functions.

```
let rec foo x =  
  function  
  | a when x = a      -> a  
  | a when x % a = 0  -> a  
  | a                 -> foo x (a + 1)  
  
let rec bar =  
  function  
  | 0 -> 0  
  | 1 -> 1  
  | x -> foo x 2  
  
let rec baz x =  
  match bar x with  
  | y when x = y -> [y]  
  | y            -> y :: (baz (x / y))
```

Question 2.1

- What do the functions `foo`, `bar`, and `baz` do? Focus on what they do rather than how they do it.
- What would be appropriate names for functions `foo`, `bar`, and `baz`.
- For these functions to behave meaningfully, we must place a restriction on the input values. What restriction?

Question 2.2

The function `foo` is slightly more complicated than it has to be. Create a function `foo2` that produces the same result as `foo` for all possible inputs but is simpler than `foo`.

Question 2.3

Create a function `baz_inverse` that takes the output of `baz` and computes the input that produced that output.

In other words, $\text{baz_inverse} (\text{baz } x) = x$ for all possible x .

Question 2.4

One of the functions from Question 2.1 is not tail recursive. Explain which one and why. To make a compelling argument you must evaluate a function call of the function, similarly to what is done in Chapter 1.4 of HR, and reason about that evaluation. You need to make clear what aspects of the evaluation tell you that the function is not tail recursive. Keep in mind that all steps in an evaluation chain must evaluate to the same value ($(5 + 4) * 3 \rightarrow 9 * 3 \rightarrow 27$, for instance).

Question 2.5

Create a function `cont` that behaves the same way as the non-tail-recursive function, that you identified in Q2.4, for all possible inputs but which is tail recursive and coded using continuations.

3: The robbers language (25%)

The robbers language is a code language made known by Astrid Lindgren's books about Kalle Blomkvist. The code works as follows. For each character in a string:

- any consonant is repeated with an `o` placed between them
- any other character is left unmodified.

For instance, the string

- Jesper becomes JoJesospoperor
- Hello World! becomes HoHelollolo WoWororloldod!

For these assignments you may, unless otherwise instructed use the following functions

```
let explode (str : string) = [for c in str -> c]
let implode (cs : char list) = cs |> Array.ofList |> System.String
let isConsonant (c : char) = "bcdfghjklmnpqrstvwxyz".IndexOf(System.Char.ToLower c) >= 0
```

Where

- `explode str` turns the string `str` into its corresponding list of characters
- `implode lst` turns a list of characters into its corresponding string
- `isConsonant c` returns `true` if `c` is an upper- or a lower-case consonant and `false` otherwise

Question 3.1

Create a non-higher-order function `encode` of type `string -> string` that given a string `str` translates `str` to the robbers language.

Examples:

```
> encode "Hello World!"
- val it: System.String = "HoHelollolo WoWororloldod!"

> encode "F# is amazing"
- val it: System.String = "FoF# isos amomazozinongog"
```

Question 3.2

Create a non-higher-order function `decode` of type `string -> string` that given a string in the robbers language `str` decodes the string into its original meaning.

Examples:

```
> decode "HoHelollolo WoWororloldod!"
- val it: System.String = "Hello World!"

> decode "FoF# isos amomazozinongog"
- val it: System.String = "F# is amazing"

> "We will rock you" |> encode |> decode
- val it: System.String = "We will rock you"
```

Question 3.3

Create a higher-order function `encode_fun` of type `(char -> string) -> string -> string` that given a function `f` and a string `str` encodes `str` by taking every character in `str` and translating it using `f`

Create the non-recursive function `encode2`, implemented using `encode_fun`, that produces the same result as `encode` for all possible inputs

Examples:

```
> encode_fun (fun c -> let cstr = string c in cstr + cstr) "Hello World!"
- val it: String = "HHeelllloo Woorrlldd!!"

> encode2 "Hello World!"
- val it: String = "HoHelollolo WoWororloldod!"
```

Question 3.4

For this assignment you may not use any of the functions you made in Q3.1 or Q3.2. You may, however, use `explode`, `implode`, or `isConsonant` if you wish.

Create a parser `parser_robbers_language` of type `Parser<string>` that consumes an input string character by character and produces its robbers language encoding.

Examples:

```
> run parser_robbers_language "Hello World!"
- val it: Result<String,string> = Ok "HoHelollolo WoWororloldod!"
```

Question 3.5

For this assignment you may use arrays as long as you use them in a functional manner — you may read from them but not mutate them.

Create a function `encode_par` of type `string -> int -> string` that given a string `str` and an integer `num` produces the same result as `encode str` for all possible `str` but which works in parallel by

- splitting `str` into a collection of the words that are separated by space
- spawning `num` parallel tasks and distributing the words as evenly as possible among the tasks
- each task encoding its words using `encode` from Q3.1
- composing the result of all tasks into one string

Hint:

- It is helpful to create a function that takes a list of words and composes them into a space-separated string
- Use `str.Split` (where `str` is a string) to split `str` into an array containing the words of `str`
- Use `List.chunkBySize` together with an appropriate chunk size to partition the words among the tasks.

4: The N-Queens problem (25%)

The Eight-queens problem is a classic chess puzzle initially posed by Max Bezzel in 1848. The objective is to place eight queens on an eight by eight chess board in such a way that no queen threatens another. Recall that in chess a queen can move any distance horizontally, vertically, and diagonally.

The following image depicts a possible solution to the problem

For this assignment we will scale this puzzle to the generalized N-Queens problem, introduced by Franz Nauck in 1850 where the standard eight by eight chess board is replaced by an N by N board for any N greater than or equal to four (technically the one by one board also has a solution).

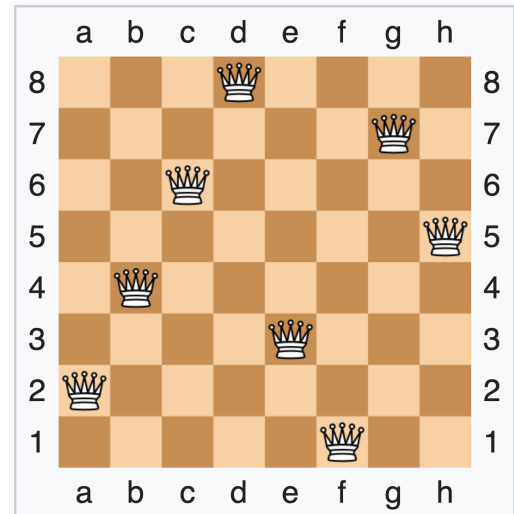
The object of this assignment is not to find a solution to the problem, but rather to check that an existing solution is correct. Make sure to read the entire question before you start as the data structure(s) you pick in Q4.1 will be instrumental in how you approach the rest of the assignment.

We will use the following notation for boards

	Q			3
			Q	2
Q				1
		Q		0
0	1	2	3	

which represents a solution for a board with $N = 4$ (four rows and four columns), where rows 0 to 3 have a queen on columns 2, 0, 3, and 1 respectively.

Important: For this assignment you may not use mutable data structures like arrays.



Source: Wikipedia

Question 4.1

Create a type `board` that contains the width of the board as well as a means to determine what squares have a queen on them.

Create a function `empty` of type `int -> board` that given an integer `N`, which you may assume is positive, creates an empty board of size `N * N`.

Create a function `get_dimension` of type `board -> int` that given a board `b` of size $N * N$ returns N .

Create a function `has_queen` of type `int -> int -> board -> bool` that given a row `r`, a column `c`, and a board `b`, returns `true` if there is a queen placed on row `r` and column `c` on the board `b` and `false` otherwise, including if either `r` or `c` are outside the board `b`.

Question 4.2

Create a function `place_queen` of type `int -> int -> board -> board option` that given a row `r`, a column `c`, and a board `b` returns

- Some b' where b' is the board b with a queen placed on the square at row r and column c if, on board b , that square is empty and placing a queen there does not threaten any other queen
- None otherwise

Create a function `valid_solution` of type `board -> bool` that given a board `b` of dimensions `N * N` returns `true` if `N` queens have successfully been placed on the board and `false` otherwise. You may assume that the only way that this solution board `b` has been created is by using the `empty` and the `place_queen` functions.

Examples:

```
> let b1 = empty 4 |> place_queen 1 0 |> Option.bind (place_queen 3 1) |> Option.bind (place_queen 0 2)
- val b1: board option =
```

	Q			3
				2
Some	Q			1
		Q		0
	0	1	2	3

```
> let b2 = b1 |> Option.bind (place_queen 2 3)
- val b2: board option =
```

		Q			3
				Q	2
Some	Q				1
			Q		0
	0	1	2	3	

```
> empty 4 |> place_queen 1 0 |> Option.bind (place_queen 0 1)
- val it: board option = None
```

```
> b1 |> Option.get |> valid_solution
- val it: bool = false
```

```
> b2 |> Option.get |> valid_solution
- val it: bool = true
```


Question 4.3

For this assignment we will be using a state monad to hide the chess board. The state monad you will be working on is very similar to the one that you used for the green assignments in your interpreter, but the state is much simpler (the `board` type from Question 4.1).

```
type chessMonad<'a> = CM of (board -> ('a * board) option)

let ret x = CM (fun h -> (Some (x, h)))
let fail = CM (fun _ -> None)
let bind f (CM a) =
  CM (fun b ->
    match a b with
    | Some (x, b') ->
      let (CM g) = f x
      g b'
    | None -> None)

let (>=) a f = bind f a
let (>>=) a b = a >= (fun _ -> b)

let evalCM (CM f) N = f (empty N)
```

Note that `evalCM` evaluates the `chessMonad` from an initial configuration of the board.

Create functions `place_queen2` of type `int -> int -> chessMonad<unit>` and `valid_solution2` of type `chessMonad<bool>` where

- `place_queen2 r c` tries to place a queen on row `r` and column `c` and fails if this is not possible
- `valid_solution2` returns `true` if the board contains a valid solution and `false` otherwise.

These functions should be implemented using `place_queen` and `valid_solution` from Q4.2 respectively, and successful results, or errors, should be returned in the state monad as appropriate. Moreover, failure means monadic failure (using `fail`) and not `failwith`.

Important: You cannot use monadic operators, like `bind` or `ret`, for `place_queen` or `valid_solution` as you must break the abstraction of the chess monad to implement these functions, but we include them here for debugging purposes, and you will need them for Question 4.4.

Examples:

```
> 4 |> evalCM (place_queen2 1 0 >>= place_queen2 3 1 >>= place_queen2 0 2 >>=
  place_queen2 2 3 >>= valid_solution2) |> Option.map fst
- val it: bool option = Some true

> 4 |> evalCM (place_queen2 1 0 >>= place_queen2 3 1 >>= place_queen2 0 2 >>= valid_solution2) |>
  Option.map fst
- val it: bool option = Some false

> 4 |> evalCM (place_queen2 1 0 >>= place_queen2 0 1 >>= valid_solution2) |> Option.map fst
- val it: bool option = None
```

Question 4.4

For this assignment you

- must use the monadic operators (one or more of `>=>`, `>>=>`, `ret` and `fail`) from Q4.3
- must not use computation expressions
- may not break the abstraction of the state monad (you may not pattern match on `CM`)

Create a function `create_solution` of type `(int * int) list -> chessMonad<bool>` that given a list of squares of the form `(r, c)`, where `r` is the row and `c` is the column, returns `true` if placing a queen on all of these squares results in a solved board and `false` otherwise.

Examples:

```
> 4 |> evalCM (create_solution [(1, 0); (3, 1); (0, 2); (2, 3)]) |> Option.map fst
- val it: bool option = Some true

> 4 |> evalCM (create_solution [(1, 0); (3, 1); (0, 2)]) |> Option.map fst
- val it: bool option = Some false

> 4 |> evalCM (create_solution [(1, 0); (0, 1)]) |> Option.map fst
- val it: bool option = None
```

Question 4.5

Create a function `create_solution2` that produces the same result as `create_solution` from Q4.4, but this time you may not use the monadic operators, or pattern match on `CM`, but rather use the following computation expression.

```
type ChessBuilder() =
  member this.Bind(f, x)    = bind x f
  member this.Return(x)    = ret x
  member this.ReturnFrom(x) = x
  member this.Combine(a, b) = a >>= (fun _ -> b)

let chess = new ChessBuilder()
```